Docker : Conteneurisation, Images et Orchestration légère

Module : Cloud Computing, Virtualisation & DevOps — Master Sciences et Techniques

Pr. A. ABASSI

Département de Physique Appliquée
Laboratoire LAVETE Lab – FST Settat

Faculté des Sciences et Techniques de Settat
Université Hassan 1er

October 21, 2026

Sommaire

1. Introduction
2. Partie I — Virtualisation vs conteneurisation
3. Partie II — Architecture de Docker
4. Partie III — Premiers pas : commandes de base
5. Partie IV — Le Dockerfile
6. Partie VI — Réseau Docker
7. Partie VII — Persistance des données : volumes
8. Partie VIII — Docker Compose
9. Partie IX — Bonnes pratiques
10. Partie X — Docker et le Cloud/DevOps
11. Étude de cas de synthèse
12. Synthèse
13. Mini-quiz
14. Corrigé du mini-exercice Dockerfile
15. TP guidé
16. Conclusion

1. Introduction

Le problème que Docker résout

La phrase la plus célèbre du DevOps

Ça marchait très bien sur ma machine !

- Une application dépend de versions précises de langages, bibliothèques, variables d'environnement.
- Ce qui fonctionne chez le développeur peut échouer sur le serveur de production (versions différentes, OS différent, dépendances manquantes).
- Installer manuellement chaque environnement est long, source d'erreurs, difficile à reproduire.

Idée de Docker

Empaqueter une application **avec tout son environnement** (code, bibliothèques, dépendances, configuration) dans une unité portable et reproductible : le **conteneur**.

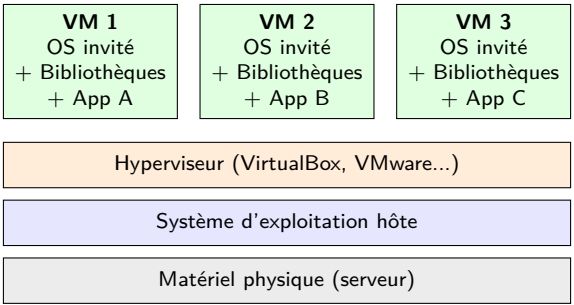
Ce que nous allons construire, pas à pas

1. Comprendre **pourquoi** un conteneur n'est pas une VM
2. Comprendre l'**architecture** de Docker
3. Manipuler les **images** et les **conteneurs**
4. Écrire un **Dockerfile** pour empaqueter sa propre application
5. Gérer le **réseau** et le **stockage** des conteneurs
6. Orchestrer plusieurs conteneurs avec **Docker Compose**
7. Appliquer les **bonnes pratiques** et relier Docker au Cloud/DevOps

2. Partie I — Virtualisation vs conteneurisation

Rappel : la virtualisation classique

Une **VM** (machine virtuelle) simule un ordinateur complet, avec son **propre système d'exploitation**, au-dessus d'un **hyperviseur** (ex. VirtualBox, VMware, KVM).



Limite

Chaque VM embarque un OS complet → lourd (plusieurs Go), lent à démarrer (minutes), gourmand en RAM/CPU.

La conteneurisation : le principe fondamental

Un **conteneur** représente une rupture radicale avec la machine virtuelle classique :

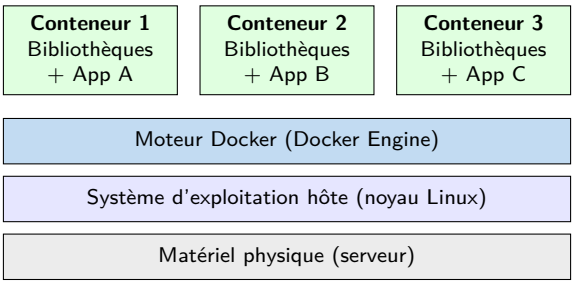
- **Absence d'OS invité dédié** : Contrairement à une VM, un conteneur ne réinstalle pas un système d'exploitation entier.
- **Partage du noyau (Kernel)** : Il s'exécute directement sur le noyau du système d'exploitation hôte.
- **Isolation logicielle poussée** : Malgré ce partage, chaque conteneur conserve son propre espace étanche pour :
  - Les processus en cours,
  - La pile réseau,
  - Le système de fichiers (via des couches dédiées).

Idée clé

On isole les applications au niveau de l'espace utilisateur, sans la lourdeur d'une virtualisation matérielle complète.

Architecture d'une plateforme de conteneurs

Visualisation de l'empilement logiciel avec Docker :



Le **Docker Engine** joue le rôle d'intermédiaire entre les conteneurs isolés et le noyau Linux sous-jacent, en pilotant les mécanismes d'isolation du noyau (namespaces et cgroups).

Pourquoi la conteneurisation séduit-elle autant ?

En évitant la duplication d'un OS par instance, cette approche apporte des gains majeurs :

- **Légèreté extrême** : Poids de l'ordre de quelques mégaoctets (au lieu de plusieurs gigaoctets pour une VM).
- **Agilité et rapidité** : Démarrage quasi instantané (en millisecondes ou secondes).
- **Haute densité matérielle** : Permet de faire tourner un nombre considérablement plus élevé de conteneurs que de machines virtuelles sur une même machine physique.

Bilan opérationnel

Des cycles de déploiement accélérés, une utilisation optimale des ressources matérielles et une portabilité totale entre postes de développement et serveurs de production.

VM vs Conteneur : tableau comparatif

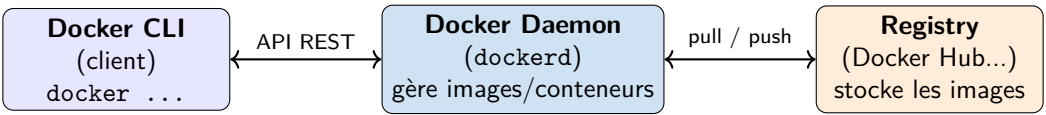
Critère	Machine virtuelle	Conteneur
Système d'exploitation	Un OS complet par VM	Partage le noyau de l'hôte
Taille	Plusieurs Go	Quelques Mo à centaines de Mo
Démarrage	Minutes	Millisecondes à secondes
Isolation	Très forte (niveau matériel)	Forte (niveau processus)
Densité	Quelques VM par serveur	Dizaines/centaines de conteneurs
Outil typique	VirtualBox, VMware, KVM	Docker, containerd

Attention

Ce ne sont pas des concurrents absolus : en pratique, on exécute très souvent des **conteneurs à l'intérieur de VM** dans le Cloud, pour cumuler les deux niveaux d'isolation.

3. Partie II — Architecture de Docker

Les trois composants de Docker



- **Docker CLI** : l'outil en ligne de commande que l'utilisateur tape (`docker run`, `docker build`...).
- **Docker Daemon** (`dockerd`) : le processus qui tourne en arrière-plan, construit les images, démarre/arrête les conteneurs.
- **Registry** : un serveur qui stocke des images Docker, prêtes à être téléchargées (*pull*) ou publiées (*push*). Le registre public par défaut est **Docker Hub**.

Image vs Conteneur : la distinction fondamentale

Image

- Un **modèle en lecture seule**
- Contient le code, les dépendances, la configuration
- Ne tourne jamais elle-même
- Comparable à une **classe** en programmation, ou à une recette de cuisine

Conteneur

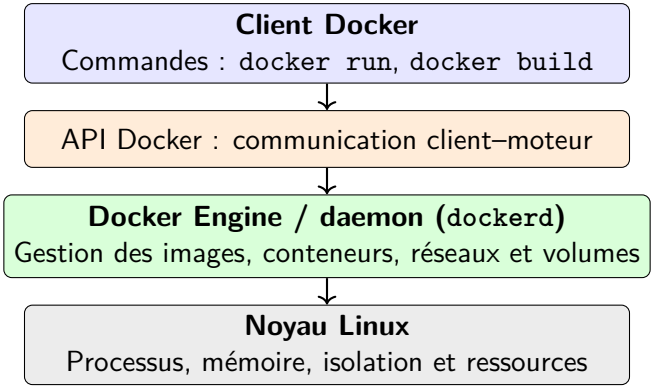
- Une **instance en cours d'exécution** d'une image
- Possède un état, des processus actifs
- Comparable à un **objet** instancié d'une classe, ou au plat préparé à partir de la recette

À retenir

On peut démarrer **plusieurs conteneurs différents** à partir de la **même image**, exactement comme on peut créer plusieurs objets à partir d'une même classe.

4. Partie III — Premiers pas : commandes de base

Architecture de Docker



- Le client envoie des commandes au moteur Docker.
- Le moteur crée et gère les conteneurs.
- Le noyau fournit les mécanismes nécessaires à l'exécution et à l'isolation des conteneurs Linux.

Docker Desktop : rôle et fonctionnement

- **Interface graphique :** visualiser les conteneurs, les images, les volumes et les journaux.
- **Environnement d'exécution :** fournit et démarre le moteur Docker nécessaire à l'exécution des conteneurs.
- **Intégration système :** configure l'environnement nécessaire à Docker selon le système d'exploitation.
- **Sous Windows :** utilise généralement WSL 2 pour exécuter les conteneurs Linux.
- **Sous Linux :** Docker Desktop est facultatif. On peut utiliser directement Docker Engine.

À retenir

Le client Docker envoie les commandes ; le moteur Docker les exécute. Docker Desktop facilite leur utilisation, mais n'est pas obligatoire sur Linux.

Installation de Docker sous Windows

1. Vérifier que Windows et le matériel sont compatibles avec Docker Desktop.
2. Activer la virtualisation matérielle dans le BIOS/UEFI si elle ne l'est pas déjà.
3. Télécharger Docker Desktop depuis : `Documentation officielle Docker Desktop`.
4. Installer Docker Desktop et privilégier le moteur basé sur WSL 2 lorsque cette option est disponible.
5. Redémarrer l'ordinateur si cela est demandé, puis lancer Docker Desktop.
6. Attendre que le moteur Docker soit opérationnel.

Point important

Installer le client Docker ne suffit pas : le moteur doit être démarré pour exécuter les conteneurs.

Installation de Docker Engine sous Ubuntu

1. Installer les outils nécessaires

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
```

2. Ajouter la cle officielle Docker

```
sudo curl -fsSL \
https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

3. Configurer le depot officiel

```
sudo tee /etc/apt/sources.list.d/docker.sources \
> /dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "\${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Installation et démarrage du moteur Docker

1. Installer Docker Engine et ses composants

```
sudo apt update
sudo apt install docker-ce docker-ce-cli \
containerd.io docker-buildx-plugin \
docker-compose-plugin
```

2. Verifier et demarrer le service

```
sudo systemctl status docker
sudo systemctl start docker
```

3. Tester l'installation

```
sudo docker run hello-world
```

Résultat attendu

Docker telecharge l'image `hello-world` si necessaire, execute un conteneur de test et affiche un message de confirmation.

Vérifier l'installation

```
docker --version
docker info
docker version
```

- `docker -version` : affiche la version du client.
- `docker info` : affiche des informations sur le moteur, les conteneurs, les images et la configuration.
- `docker version` : affiche les versions du client et du serveur Docker, lorsque le moteur est accessible.

Tester l'exécution d'un conteneur

```
docker run hello-world
```

Cette commande telecharge l'image si necessaire, cree un conteneur de test et affiche un message de confirmation.

`docker run` : démarrer un conteneur

```
docker run ubuntu
```

- Si l'image `ubuntu` n'est pas déjà présente localement, Docker la **télécharge** automatiquement depuis Docker Hub (*pull* implicite).
- Un **nouveau conteneur** est créé à partir de cette image, puis démarré.
- Sans commande interactive, le conteneur s'exécute puis s'arrête immédiatement (rien à faire).

```
docker run -it ubuntu bash
```

- `-i` (*interactive*) : garde l'entrée standard ouverte.
- `-t` (*tty*) : alloue un terminal.
- `bash` : commande exécutée dans le conteneur → ouvre un shell interactif à l'intérieur.

Options courantes de `docker run`

```
docker run -d --name monweb -p 8080:80 nginx
```

- `-d` (*detached*) : exécute le conteneur en **arrière-plan**, sans bloquer le terminal.
- `-name monweb` : attribue un nom lisible au conteneur (sinon Docker en génère un aléatoire).
- `-p 8080:80` : redirige le port **80 du conteneur** vers le port **8080 de l'hôte** (voir Partie VI).
- `nginx` : l'image utilisée, ici un serveur web.

Question de vérification

Une fois cette commande exécutée, quelle URL faut-il ouvrir dans un navigateur sur la machine hôte pour accéder au serveur Nginx ?

Lister, inspecter, arrêter

```
docker ps
docker ps -a
```

`docker ps` liste les conteneurs **en cours d'exécution**. L'option `-a` (*all*) affiche aussi les conteneurs arrêtés.

```
docker stop monweb
docker start monweb
docker restart monweb
```

Arrête, redémarre un conteneur **existant** (sans le supprimer) — `monweb` peut être remplacé par l'ID ou le nom du conteneur.

```
docker rm monweb
```

Supprime un conteneur **arrêté** (un conteneur en cours d'exécution doit d'abord être arrêté, ou supprimé avec `-f`).

Gérer les images

```
docker images
```

Liste toutes les images présentes localement.

```
docker pull nginx:1.25
```

Télécharge explicitement une image depuis un registre, ici la version `1.25` de Nginx (le **tag** après les deux-points précise la version ; sans tag, Docker utilise `latest`).

```
docker rmi nginx:1.25
```

Supprime une image locale (impossible si un conteneur l'utilise encore).

Attention

Un tag `latest` ne signifie pas ń la dernière version stable ż de façon garantie : c'est juste le tag par défaut choisi par le mainteneur de l'image. En production, on fige toujours une version précise.

Explorer et déboguer un conteneur en cours d'exécution

```
docker exec -it monweb bash
```

Ouvre un shell **à l'intérieur** d'un conteneur déjà démarré (sans le redémarrer) — très utile pour inspecter un problème.

```
docker logs monweb
docker logs -f monweb
```

Affiche les journaux (sortie standard) du conteneur. `-f` (*follow*) suit les journaux en temps réel, comme `tail -f`.

```
docker inspect monweb
```

Affiche, au format JSON, tous les détails de configuration du conteneur (IP interne, volumes, variables d'environnement...).

Mini-exercice – Manipuler un conteneur Redis

Contexte :
Redis est un logiciel qui permet de stocker des données rapidement. Dans cet exercice, nous allons utiliser une image Docker appelée `redis`, sans étudier son fonctionnement interne.

Travail demandé :

1. **Démarrer un conteneur :**
   À partir de l'image `redis`, écrire la commande permettant de créer et de démarrer un conteneur nommé `cache01` en arrière-plan.

2. **Lister les conteneurs :**
   Écrire la commande permettant d'afficher tous les conteneurs Docker, qu'ils soient démarrés ou arrêtés.

3. **Accéder au conteneur :**
   Écrire la commande permettant d'ouvrir un terminal Bash à l'intérieur du conteneur `cache01` déjà démarré.

Indication : Vous pouvez utiliser les commandes `docker run`, `docker ps` et `docker exec`.

5. Partie IV — Le Dockerfile

Objectifs de cette partie

À la fin de cette partie, l'étudiant sera capable de :

- expliquer le rôle d'un Dockerfile ;
- créer une petite application Python ;
- préparer un Dockerfile étape par étape ;
- construire une image Docker ;
- créer et exécuter un conteneur à partir de cette image ;
- comprendre les principales instructions Docker ;
- identifier et corriger des erreurs simples.

Prérequis

Aucune connaissance préalable du Dockerfile n'est nécessaire. Il suffit de connaître les bases de Python et les commandes simples du terminal.

1. Pourquoi avons-nous besoin d'un Dockerfile ?

Imaginons que nous avons écrit un programme Python.

```
print("Bonjour, bienvenue dans Docker !")
```

Ce programme fonctionne sur notre ordinateur si Python est installé.
Mais que se passe-t-il si nous voulons exécuter ce programme sur un autre ordinateur qui ne possède pas Python ?

Nous devons normalement :

1. installer Python ;
2. vérifier sa version ;
3. installer les bibliothèques nécessaires ;
4. copier le code du programme ;
5. lancer le programme.

Le problème

Ces opérations peuvent être longues et produire des différences entre les environnements des développeurs.

2. L'idée du Dockerfile

Au lieu de répéter manuellement les opérations précédentes, nous pouvons les décrire dans un fichier texte.
Ce fichier s'appelle **Dockerfile**.

Définition

Un Dockerfile est un fichier texte contenant des instructions que Docker utilise pour construire une image.

Cette image peut ensuite servir à créer un ou plusieurs conteneurs.



- **Dockerfile :** la recette de fabrication.
- **Image :** le résultat construit à partir de cette recette.
- **Conteneur :** une instance de l'image en cours d'exécution.

3. Préparer notre premier projet

Nous allons créer un projet appelé `premier-projet-docker`.
Créons un dossier portant ce nom sur notre ordinateur.
À l'intérieur, créons un fichier appelé `app.py`.

Organisation du projet

```
premier-projet-docker/
app.py
```

Pour le moment, nous avons seulement un programme Python.

Attention : le fichier doit s'appeler exactement `app.py`, et non `app.py.txt`.

Sous Windows

Dans l'Explorateur de fichiers, activez l'affichage des extensions si nécessaire, afin de vérifier le nom réel du fichier.

4. Étape 1 : écrire le programme Python

Ouvrons le fichier `app.py` et écrivons le code suivant.

```
print("Bonjour !")
print("Je decouvre Docker.")
print("Mon premier conteneur fonctionne.")
```

Explication du code :

- `print(...)` affiche un message dans le terminal.
- Chaque instruction affiche un message différent.
- Ce programme ne nécessite aucune bibliothèque externe.

Test avant Docker :

Si Python est installé, ouvrons un terminal dans le dossier du projet et exécutons :

```
python app.py
```

Sur certains systèmes, la commande est :

```
python3 app.py
```

Résultat attendu

5. Étape 2 : préparer Docker

Avant de construire notre image, nous devons disposer de Docker.

- Sous Windows ou macOS, nous pouvons utiliser Docker Desktop.
- Sous Linux, nous pouvons installer Docker Engine.

Après l'installation, ouvrons un terminal et exécutons :

```
docker --version
```

Cette commande affiche la version du client Docker.

Exécutons ensuite :

```
docker info
```

Cette seconde commande affiche des informations sur Docker, notamment sur le moteur qui gère les images et les conteneurs.

Si une erreur apparaît

Si Docker indique que le moteur ne fonctionne pas, vérifiez que Docker Desktop est démarré ou que le service Docker est actif sous Linux.

6. Étape 3 : créer notre premier Dockerfile

Dans le même dossier que `app.py`, créons un fichier nommé exactement `Dockerfile`.

Structure du dossier

```
premier-projet-docker/
app.py
Dockerfile
```

Écrivons une seule instruction dans le Dockerfile :

```
FROM python:3.11-slim
```

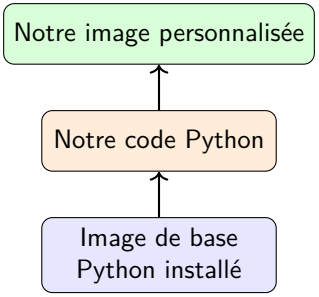
Que signifie cette instruction ?

- `FROM` signifie que nous partons d'une image existante.
- `python` désigne l'image qui fournit Python.
- `3.11` indique la version de Python souhaitée.
- `slim` désigne une variante allégée de cette image.

Idée essentielle

7. Comprendre FROM grâce à une analogie

Imaginons que nous voulons préparer une salle informatique.
Nous avons besoin d'un bâtiment avant d'installer les équipements.



L'image de base est notre point de départ.
Nous ajouterons ensuite notre code et, si nécessaire, les bibliothèques utilisées par notre application.

À retenir : `FROM` ne lance pas notre programme. Il indique simplement l'image de départ utilisée pour la construction.

8. Étape 4 : ajouter WORKDIR

Complétons notre Dockerfile :

```
FROM python:3.11-slim

WORKDIR /app
```

Que fait `WORKDIR /app` ?

- `WORKDIR` définit le répertoire de travail dans l'image.
- `/app` est le chemin choisi pour notre application.
- Si ce répertoire n'existe pas, Docker le crée.

Nous pouvons imaginer que Docker prépare un espace de travail dans lequel nous placerons les fichiers de notre projet.

Important

Le dossier `/app` se trouve dans l'environnement de l'image. Ce n'est pas automatiquement le dossier app de Windows.

Les instructions suivantes, notamment `COPY` et certaines commandes `RUN`, utiliseront ce répertoire comme référence lorsque leur chemin est relatif.

9. Étape 5 : copier notre programme

Ajoutons une nouvelle instruction :

```
FROM python:3.11-slim

WORKDIR /app

COPY app.py .
```

Comprenons la commande :

```
COPY app.py .
```

- `COPY` demande à Docker de copier un fichier.
- `app.py` est le fichier source présent dans le contexte de construction.
- `.` désigne ici le répertoire de destination courant, c'est-à-dire `/app`.

Après cette instruction, le fichier se trouve à l'emplacement :

`/app/app.py`

Ne pas confondre

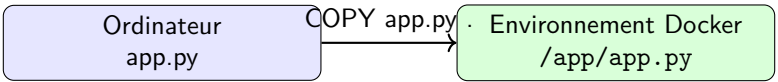
Le fichier source est recherché dans le contexte de construction, pas dans un répertoire

10. Comprendre le chemin de COPY

Supposons que notre projet soit enregistré sur Windows dans :

`C:\Cours\premier-projet-docker`

Nous ouvrons un terminal dans ce dossier et construisons l'image.



- Le fichier original reste sur notre ordinateur.
- Docker en copie une version dans l'image.
- Le programme du conteneur utilisera cette copie.

Conséquence

Modifier le fichier sur l'ordinateur ne modifie pas automatiquement le fichier déjà intégré dans une image construite. Il faudra reconstruire l'image pour intégrer cette modification.

11. Étape 6 : construire l'image Docker

Nous avons maintenant notre premier Dockerfile complet :

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
```

Ouvrons le terminal dans le dossier du projet et exécutons :

```
docker build -t premier-python:1.0 .
```

Décomposons la commande :

- `docker build` demande à Docker de construire une image.
- `-t` permet de donner un nom et une étiquette à l'image.
- `premier-python` est le nom choisi.
- `1.0` est la version indiquée par le tag.
- `.` indique le dossier courant comme contexte de construction.

Docker lit le Dockerfile, récupère l'image de base si nécessaire, puis prépare l'image personnalisée.

12. Vérifier que l'image a été construite

Après une construction réussie, exécutons :

```
docker images
```

Cette commande affiche les images disponibles localement.
Nous devons notamment retrouver une image nommée :

`premier-python`

avec le tag :

`1.0`

Nous pouvons également utiliser :

```
docker image ls
```

Attention

Une image n'est pas un programme en cours d'exécution. À ce stade, nous avons construit l'image, mais nous n'avons pas encore lancé notre application.

13. Étape 7 : créer un conteneur

Pour créer un conteneur à partir de notre image, exécutons :

```
docker run --name test-python premier-python:1.0
```

Explication :

- `docker run` crée un nouveau conteneur à partir de l'image, puis démarre le processus prévu.
- `-name test-python` donne un nom au conteneur.
- `premier-python:1.0` désigne l'image à utiliser.

Le programme affiche les trois messages, puis se termine.

Pourquoi le conteneur s'arrête-t-il ?

Notre programme affiche des messages puis termine son exécution. Lorsque son processus principal se termine, le conteneur s'arrête. Cela ne signifie pas que l'image a été supprimée.

14. Comprendre la commande docker run

Comparons les commandes suivantes.

```
docker run premier-python:1.0
```

Docker crée un conteneur et exécute le programme.

```
docker run --rm premier-python:1.0
```

Le programme est exécuté de la même manière, mais Docker supprime automatiquement le conteneur lorsqu'il se termine.

```
docker ps
```

Affiche les conteneurs en cours d'exécution.

```
docker ps -a
```

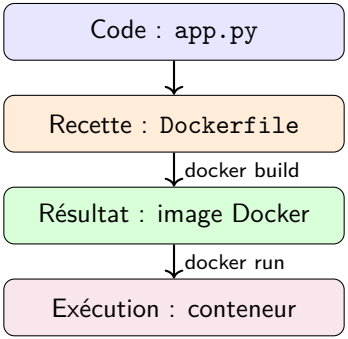
Affiche également les conteneurs arrêtés.

À retenir

L'image peut être réutilisée pour créer plusieurs conteneurs. La suppression d'un conteneur ne

15. Bilan : notre première image

Nous avons construit une image sans écrire beaucoup de code.



Les quatre éléments à retenir

- `app.py` contient le programme.
- `Dockerfile` décrit comment préparer l'image.
- `docker build` construit l'image.

16. Pourquoi ajouter de nouvelles instructions ?

Notre première version contient :

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
```

Elle fournit Python et copie le programme dans l'image.
Cependant, nous souhaitons également :

- définir une commande de démarrage par défaut ;
- installer les bibliothèques nécessaires, si notre programme en utilise ;
- documenter le port réseau, pour une application Web ;
- organiser la construction de manière efficace.

Notre méthode

Nous ajoutons les instructions progressivement et nous expliquons leur rôle avant de construire une nouvelle image.

17. Étape 8 : ajouter CMD

Modifions notre Dockerfile :

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

Examinons la dernière ligne :

```
CMD ["python", "app.py"]
```

- `CMD` définit la commande exécutée par défaut au démarrage du conteneur.
- `python` désigne l'interpréteur Python.
- `app.py` désigne le fichier à exécuter.

Cette instruction signifie donc :

Lorsque le conteneur démarre, exécuter le programme app.py avec Python.

Important

`CMD` ne construit pas l'image et n'exécute pas le programme pendant le build. La commande est exécutée au démarrage du conteneur.

18. Tester CMD

Reconstruisons l'image :

```
docker build -t premier-python:2.0 .
```

Puis exécutons :

```
docker run --rm premier-python:2.0
```

Nous obtenons les messages de notre programme sans avoir besoin de préciser la commande Python.

Comparons avec la commande suivante :

```
docker run --rm premier-python:2.0 python -c "print('Test')"
```

Ici, la commande fournie après le nom de l'image remplace le `CMD` par défaut.

À retenir

Grâce à `CMD`, l'utilisateur n'a normalement pas besoin de connaître la commande exacte pour démarrer l'application.

19. Comprendre les instructions déjà utilisées

Instruction	Rôle	Exemple
`FROM`	Choisir l'image de base	`FROM python:3.11-slim`
`WORKDIR`	Définir le dossier de travail	`WORKDIR /app`
`COPY`	Copier des fichiers dans l'image	`COPY app.py .`
`CMD`	Définir la commande par défaut	`CMD ["python","app.py"]`

Ordre de lecture

Docker lit les instructions du Dockerfile de haut en bas. L'ordre est important : par exemple, il faut choisir l'image de base avant de copier notre application dans celle-ci.

20. Et si notre application utilise une bibliothèque ?

Jusqu'à présent, notre programme utilise seulement la fonction `print()`, qui fait partie de Python.
Imaginons maintenant que nous développons une application qui utilise une bibliothèque externe, par exemple Flask.

Pour exécuter cette application, Python seul ne suffit pas : Flask doit également être installé.

Deux possibilités

1. Installer Flask manuellement dans l'environnement.
2. Décrire son installation dans le projet et automatiser cette opération pendant la construction de l'image.

Nous allons choisir la deuxième méthode, car elle permet de reproduire plus facilement l'environnement de l'application.

21. Étape 9 : créer requirements.txt

Dans le dossier du projet, créons un nouveau fichier :

`requirements.txt`

Pour notre futur exemple Web, écrivons :

```
Flask==3.1.2
```

Que signifie cette ligne ?

- `Flask` est le nom de la bibliothèque Python.
- `==` indique une version précise.
- `3.1.2` est la version demandée.

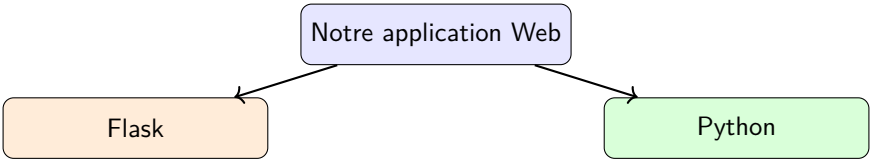
Ce fichier sert à indiquer à l'outil d'installation Python quelles bibliothèques doivent être installées.

Attention

Le fichier `requirements.txt` ne contient pas le code de notre application. Il décrit ses dépendances Python.

22. Comprendre le mot dépendance

Une dépendance est un composant dont notre application a besoin pour fonctionner.
Par exemple :



Notre application utilise Flask pour recevoir et traiter les requêtes HTTP. Flask, à son tour, s'exécute dans l'environnement Python.

Pourquoi déclarer les dépendances ?

Pour qu'un autre ordinateur puisse reconstruire un environnement comparable, sans devoir deviner quelles bibliothèques installer.

23. Étape 10 : copier requirements.txt

Notre projet contient maintenant deux fichiers :

```
premier-projet-docker/
app.py
requirements.txt
Dockerfile
```

Ajoutons la ligne suivante au Dockerfile :

```
COPY requirements.txt .
```

Elle copie le fichier `requirements.txt` du contexte de construction vers le dossier de travail de l'image.
Puisque nous avons défini :

```
WORKDIR /app
```

le fichier sera placé à l'emplacement :

`/app/requirements.txt`

24. Étape 11 : comprendre RUN

Ajoutons maintenant cette instruction :

```
RUN pip install --no-cache-dir -r requirements.txt
```

Décomposons-la :

- `RUN` exécute une commande pendant la construction de l'image.
- `pip` est l'outil qui installe les bibliothèques Python.
- `install` demande l'installation des bibliothèques.
- `-r requirements.txt` demande de lire la liste des dépendances dans ce fichier.
- `-no-cache-dir` évite de conserver le cache de téléchargement de pip dans l'image.

Résultat

Lorsque la construction se termine correctement, l'image contient les bibliothèques installées par cette commande.

25. RUN et CMD : une différence fondamentale

Comparons ces deux instructions.

RUN

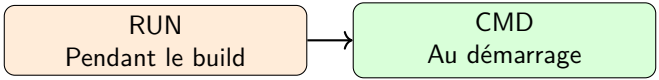
`RUN pip install -r requirements.txt`

Cette commande s'exécute pendant la construction de l'image. Elle prépare l'environnement de l'application.

CMD

`CMD ["python", "app.py"]`

Cette commande définit le processus lancé par défaut lorsque nous démarrons un conteneur à partir de l'image.



Règle simple : RUN prépare l'environnement ; CMD lance le programme par défaut.

26. Étape 12 : comprendre COPY . .

Vous avez déjà rencontré :

```
COPY app.py .
```

Cette commande copie uniquement `app.py`.
Pour copier tous les fichiers et dossiers du contexte de construction qui ne sont pas exclus, nous pouvons utiliser :

```
COPY . .
```

Les deux points représentent deux emplacements :

- le premier `.` représente la source : le contexte de construction ;
- le deuxième `.` représente la destination : le répertoire de travail défini par `WORKDIR`.

Dans notre projet, nous pouvons écrire :

```
WORKDIR /app
COPY . .
```

Les fichiers du projet sont alors copiés dans `/app`.

Bonne pratique

27. Pourquoi copier requirements.txt avant le code ?

Considérons cette organisation :

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

Imaginons que nous modifions seulement le texte affiché par `app.py`.

- Le fichier `requirements.txt` n'a pas changé.
- La commande d'installation n'a pas besoin d'être répétée si Docker peut réutiliser la couche correspondante.
- Le code modifié est copié lors de la reconstruction.

Le cache de construction

Docker peut réutiliser les résultats d'instructions déjà construites lorsque les conditions de réutilisation du cache sont satisfaites. Cela peut accélérer les constructions suivantes.

Attention

Si `requirements.txt` change, Docker doit normalement recalculer l'installation des dépendances.

28. Notre Dockerfile avec dépendances

Nous pouvons maintenant réunir les instructions étudiées :

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

CMD ["python", "app.py"]
```

Lecture du Dockerfile dans l'ordre :

1. Choisir une image contenant Python.
2. Définir le répertoire de travail.
3. Copier la liste des dépendances.
4. Installer ces dépendances.
5. Copier le programme.
6. Définir la commande de démarrage.

29. Qu'est-ce que Flask ?

Jusqu'à présent, notre programme Python affiche des messages dans le terminal.
Nous souhaitons maintenant créer une petite application Web.

Définition

Flask est une bibliothèque Python permettant de développer des applications Web. Elle permet notamment de définir les adresses auxquelles un navigateur peut envoyer des requêtes.

Par exemple, lorsque l'utilisateur ouvre :

```
http://localhost:5000
```

notre application pourra répondre avec un message.

Important : Flask n'est pas intégré à Python par défaut. Nous devons l'installer comme une dépendance.

30. Étape 13 : créer une application Flask

Remplaçons le contenu de `app.py` par le code suivant :

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def accueil():
return "Bonjour ! Mon application fonctionne."

if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Ce code est volontairement petit. Examinons-le instruction par instruction dans les diapositives suivantes.

Notre objectif

Le navigateur doit pouvoir afficher le message renvoyé par la fonction `accueil()`.

31. Comprendre l'importation et la création de Flask

Première ligne :

```
from flask import Flask
```

Cette instruction importe la classe `Flask` depuis la bibliothèque installée.
Deuxième instruction :

```
app = Flask(__name__)
```

- `Flask(...)` crée l'application Web.
- app est la variable qui référence cette application.
- `__name__` fournit à Flask des informations sur le module Python courant.

Analogie

La variable app représente notre application Web. Nous l'utiliserons pour définir ses routes et démarrer le serveur.

32. Comprendre une route Flask

Examinons les instructions suivantes :

```
@app.route("/")
def accueil():
return "Bonjour ! Mon application fonctionne."
```

- `@app.route("/")` associe la racine `/` de l'application à la fonction qui suit.
- `def accueil():` définit une fonction Python nommée `accueil`.
- `return ...` renvoie le contenu de la réponse HTTP.

Lorsque le navigateur demande la page d'accueil, Flask appelle cette fonction et renvoie son message.

Résultat attendu

La page affiche : ń Bonjour ! Mon application fonctionne. ż

33. Comprendre host et port

Dernière instruction :

```
app.run(host="0.0.0.0", port=5000)
```

- `app.run(...)` démarre le serveur de développement de Flask.
- `host="0.0.0.0"` permet au serveur d'écouter sur toutes les interfaces réseau du conteneur.
- `port=5000` choisit le port d'écoute dans le conteneur.

Pourquoi 0.0.0.0 ?

Dans un conteneur, écouter uniquement sur `127.0.0.1` limiterait l'accès à l'interface de boucle locale du conteneur. Pour accéder au serveur via une redirection de port Docker, il faut normalement écouter sur une interface accessible du conteneur.

À noter

Le serveur intégré à Flask convient aux exercices pédagogiques. Pour un déploiement de production, on utilise généralement un serveur WSGI adapté.

34. Tester Flask sans Docker

Avant de conteneuriser notre application, nous pouvons la tester directement sur l'ordinateur. Dans un terminal ouvert dans le dossier du projet, installons la dépendance si nécessaire :

```
python -m pip install -r requirements.txt
```

Puis lançons :

```
python app.py
```

Si le serveur démarre correctement, ouvrons un navigateur et saisissons :

`http://localhost:5000`

Résultat

Le navigateur doit afficher le message de notre application.

Si le port est déjà utilisé

Un autre programme peut déjà utiliser le port 5000. Dans ce cas, arrêtez ce programme ou choisissez un autre port pour l'exercice.

35. Étape 14 : préparer le Dockerfile Flask

Notre dossier contient maintenant :

```
premier-projet-docker/
app.py
requirements.txt
Dockerfile
```

Voici le Dockerfile complet :

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

36. Comprendre EXPOSE 5000

Notre application Flask écoute sur le port 5000 à l'intérieur du conteneur.
Le Dockerfile contient :

```
EXPOSE 5000
```

Cette instruction indique le port sur lequel l'application est censée recevoir des connexions.

Attention

`EXPOSE 5000` ne rend pas, à lui seul, l'application accessible depuis le navigateur de l'ordinateur hôte.

Pour publier le port du conteneur sur l'ordinateur, nous utiliserons une option de `docker run` :

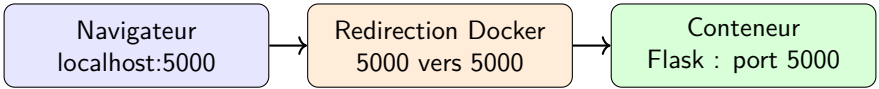
```
-p 5000:5000
```

Le premier nombre correspond au port de l'ordinateur hôte ; le deuxième correspond au port du conteneur.

37. Comprendre la redirection des ports

Considérons la commande :

```
docker run -p 5000:5000 monapp:1.0
```



- Le navigateur contacte le port 5000 de l'ordinateur.
- Docker redirige la connexion vers le port 5000 du conteneur.
- Flask reçoit la requête et renvoie sa réponse.

À retenir

Le port publié et le port d'écoute de l'application sont deux notions différentes, même si nous utilisons ici le même numéro.

38. Étape 15 : construire l'image Flask

Nous pouvons maintenant construire l'image de notre application Web.
Dans le terminal, depuis le dossier contenant le Dockerfile :

```
docker build -t monapp-flask:1.0 .
```

Que fait Docker pendant cette opération ?

1. Il lit le Dockerfile.
2. Il récupère l'image Python si nécessaire.
3. Il prépare le répertoire `/app`.
4. Il copie `requirements.txt`.
5. Il installe Flask avec pip.
6. Il copie `app.py`.
7. Il enregistre les paramètres de démarrage.

Si une étape échoue, Docker affiche une erreur et la construction ne se termine pas correctement.

39. Étape 16 : démarrer l'application Web

Une fois l'image construite, exécutons :

```
docker run --name mon-serveur -p 5000:5000 monapp-flask:1.0
```

Cette commande :

- crée un conteneur nommé `mon-serveur` ;
- publie le port 5000 du conteneur sur le port 5000 de l'hôte ;
- démarre la commande définie dans `CMD`.

Ouvrons ensuite le navigateur à l'adresse :

`http://localhost:5000`

Résultat attendu

Le navigateur affiche le message renvoyé par la fonction `accueil()`.

40. Arrêter et supprimer le conteneur

Lorsque le serveur fonctionne dans un terminal au premier plan, nous pouvons l'arrêter avec `Ctrl+C`.

Nous pouvons également ouvrir un autre terminal et exécuter :

```
docker stop mon-serveur
```

Pour supprimer ensuite le conteneur arrêté :

```
docker rm mon-serveur
```

Pour vérifier les conteneurs existants :

```
docker ps -a
```

Ne pas confondre

`docker stop` arrête le conteneur. `docker rm` supprime le conteneur. Aucune de ces commandes ne supprime automatiquement l'image.

41. Comprendre le contexte de construction

Lorsque nous exécutons :

```
docker build -t monapp-flask:1.0 .
```

le point final a une signification importante.
Il indique à Docker que le dossier courant constitue le **contexte de construction**.

Qu'est-ce que le contexte de construction ?

C'est l'ensemble des fichiers et dossiers que Docker peut utiliser comme sources pour les instructions de copie, sous réserve des règles d'exclusion et des restrictions de chemin.

Par exemple, si le terminal est ouvert dans :

```
premier-projet-docker/
Dockerfile
app.py
requirements.txt
```

la commande `docker build ... .` utilise ce dossier comme contexte.

42. Comprendre le fichier .dockerignore

Dans un projet réel, le dossier peut contenir des fichiers inutiles à la construction de l'image. Créons un fichier nommé exactement `.dockerignore` :

```
__pycache__/
*.pyc
.git/
.env
.venv/
venv/
```

Rôle de chaque règle :

- `__pycache__/` : exclut les caches Python.
- `*.pyc` : exclut les fichiers Python compilés.
- `.git/` : exclut les données du dépôt Git.
- `.env` : exclut le fichier de configuration local.
- `.venv/` et `venv/` : excluent les environnements virtuels locaux.

Pourquoi l'utiliser ?

Il permet de réduire le contexte transmis à Docker et d'éviter d'inclure inutilement certains fichiers dans le processus de construction.

43. Attention aux fichiers sensibles

Un fichier `.env` peut contenir des informations sensibles : mots de passe, clés API ou paramètres de connexion.
Il est donc généralement préférable de l'exclure du contexte de construction.

Point important

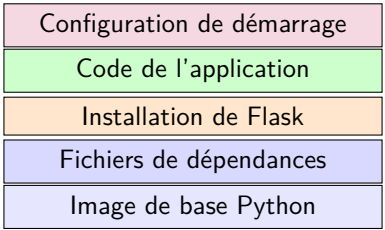
`.dockerignore` n'est pas un système de gestion des secrets. Il faut aussi éviter de transmettre des secrets par d'autres moyens dans les instructions de construction ou dans l'image.

Exemple

Pour une application pédagogique, un fichier `.env` local peut contenir des paramètres de développement. Il ne faut pas copier automatiquement ce fichier dans l'image.
Pour un déploiement, les secrets doivent être fournis par un mécanisme adapté à l'environnement d'exécution.

44. Comprendre les couches d'une image

Docker construit l'image à partir de plusieurs étapes. Certaines instructions produisent des couches de système de fichiers, tandis que d'autres définissent les métadonnées.



Pourquoi est-ce utile ?

Docker réutilise les résultats des étapes déjà construites, évitant de tout recommencer depuis le début.

Remarque : Toutes les instructions ne créent pas une couche de fichiers (ex. `CMD` définit les paramètres de démarrage).

45. Le cache de construction : exemple concret

Considérons ces instructions :

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
```

Situation A : nous modifions uniquement le texte affiché dans `app.py`.
Docker peut réutiliser le résultat de l'installation de Flask.

Situation B : nous ajoutons une nouvelle bibliothèque dans `requirements.txt`.
Docker doit normalement réexécuter l'installation des dépendances.

Conclusion

Nous plaçons généralement les dépendances avant le code applicatif, car les dépendances changent souvent moins fréquemment que le code.

46. RUN, CMD et ENTRYPOINT

Ces trois instructions ont des rôles différents.

Instruction	Moment	Rôle
`RUN`	Construction	Exécuter une commande pour préparer l'image.
`CMD`	Démarrage	Définir la commande ou les arguments par défaut.
`ENTRYPOINT`	Démarrage	Définir le programme principal à exécuter.

Exemples

`RUN pip install flask` installe une bibliothèque pendant la construction.
`CMD ["python", "app.py"]` définit la commande par défaut.
`ENTRYPOINT ["python"]` définit Python comme programme principal du conteneur.

47. CMD et ENTRYPOINT : exemple

Imaginons ce Dockerfile :

```
FROM python:3.11-slim
ENTRYPOINT ["python"]
CMD ["app.py"]
```

- **Sans argument :** `docker run monimage:1.0` → exécute `python app.py`
- **Avec argument :** `docker run monimage:1.0 autre.py` → remplace le `CMD` et exécute `python autre.py`

À retenir

`ENTRYPOINT` fixe le programme principal, tandis que `CMD` fournit les arguments par défaut modifiables.

48. Erreurs fréquentes des débutants

1. **Nom incorrect du fichier**
   Le fichier doit s'appeler `Dockerfile`, sauf si un autre nom est explicitement indiqué à Docker.
2. **Oubli du contexte de construction**
   Utiliser `docker build -t monapp:1.0 .` depuis le bon dossier.
3. **Fichier absent**
   Vérifier que les fichiers mentionnés dans `COPY` existent dans le contexte.
4. **Dépendance non installée**
   Vérifier `requirements.txt` et la commande `RUN`.
5. **Port non publié**
   Pour accéder à une application Web depuis l'hôte, vérifier la redirection avec `-p`.
6. **Application inaccessible**
   Vérifier que le serveur écoute sur une interface adaptée et sur le port attendu.

49. Exercice : compléter un Dockerfile

On dispose de l'organisation suivante :

```
mon-projet/
app.py
requirements.txt
Dockerfile
```

Le fichier `requirements.txt` contient :

```
Flask==3.1.2
```

Compléter le Dockerfile afin de :

1. utiliser une image Python 3.11 allégée ;
2. définir `/app` comme répertoire de travail ;
3. copier le fichier des dépendances ;
4. installer les dépendances ;
5. copier le code de l'application ;
6. documenter le port 5000 ;

50. Corrigé de l'exercice

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Pour construire l'image :

```
docker build -t monapp-flask:1.0 .
```

Pour démarrer le conteneur :

```
docker run --name mon-serveur \
-p 5000:5000 monapp-flask:1.0
```

Pour tester l'application, ouvrir :